Adventures and Thoughts

while implementing from scratch

Parquet

Apache orc™

BigData Formats

FABIAN GROFFEN

About me

- PhD 2009, CWI MonetDB
- Hyves 2008 (Always in touch with your friends!)
- Booking.com 2013
- Oracle Database 2016
- Gentoo developer since 2004 (still compiling…)

- NOT affiliated with Apache, Parquet, ORC, Avro etc.
- NOT talking about Oracle today :)

How it all began

- At Oracle, someone asked, “can you do a Parquet reader, in C”
- So I did.

- And then I did Avro and ORC too
  - MonetDB Columnar background

- The amazing story of how you can always overcomplicate doing something simple

What will I be talking about

- How do the formats Parquet, Avro and ORC work (internally)
- Anecdotes of how redi^Winteresting some of these formats are
- My opinion of why some things are smart, and some … errr, not so
- The quest for performance, as ever

- *Again, I will NOT be talking about Oracle, Oracle Database's internals or specifics*
- *All of my expressions and opinions are my own, and not those of Oracle (my current employer)*

Parquet

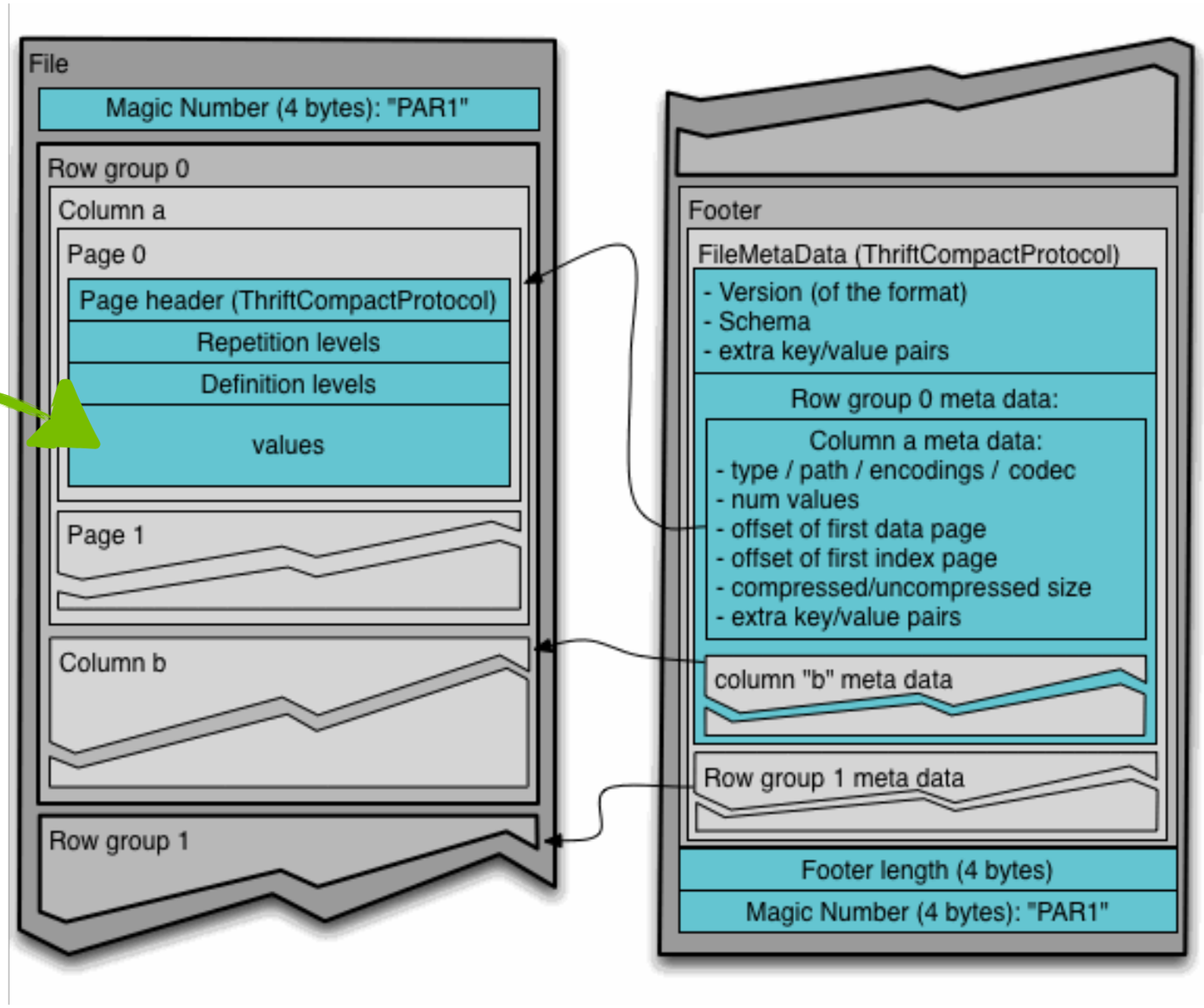
- For some reason the no 1 BigData format: so it must be really well thought out, no?
- Used by Cloudera Hadoop offering
- Binary format, as opposed to all text like CSV
- Types like strings, integers, doubles, decimals, timestamps
- Columnar (or called like that)
- Built in (homegrown) encodings/compressions

Parquet structure

- MetaData block describing the data (columns, positions, etc.)
- Data is stored in RowGroups (like a vector)
- Each RowGroup contains ColumnChunks (column(ar) data)
- Each ColumnChunk contains data blocks stored back to back
- Each data block consists of a PageHeader + Data
- RowGroups come with statistics to allow skipping based on values

Parquet file layout

MetaData is stored at the back of the file!



to read **any** value, you have to traverse like 7 layers of indirections!

About some Parquet sizes

writer dictates how much memory a reader needs!

- 3 units: file size, RowGroup size and Page size
- RowGroup size at some point was HDFS-aligned, 128MB
  - currently sizes like 4GB can be seen
- Page size usually is somewhere around 10MB, but nothing restricts it
- File size is less of an issue, main processing unit is a RowGroup

most users do not understand any of these sizes at all

- FileMetaData size cannot be guessed accurately from file size :)

so a reader must always read the entire object or attempt to read a bit from the end, hoping it is sufficient (networked reads are slow)

Reading Parquet metadata

- Stored using Thrift, binary protocol meant for RPCs
- ThriftCompactProtocol aims to be “compact” (e.g. use less bytes)
- Supports lists, numbers, strings, etc. encoded using varints
- Dynamic, like JSON, using keys addressed via an offset
- Reader and writer must use the same schema (which unlike JSON is not transported)

```
struct Phone {
  1: i32 id,
  2: string number,
  3: PhoneType type
}
```

lots of single byte reads, expansions (ints), are the few saved bytes really worth the complexity?

Parquet encodings

it is usual to first do an encoding (e.g. dictionary) and then compression, where is the point in that?

- Data Page in Parquet can be:
  - compressed (Snappy, GZIP, Zstandard, etc.)
  - encoded (plain, dictionaries, bitpacking, run length and more)
  - encrypted
  - *combination of the ones above*
- NULL and nested position are encoded using Dremel in repetition and definition levels

these are bitpacked lists of values stored for an entire page, and nested value reconstruction is very complicated (at least to me) and containing a lot of redundant information

Columnar Parquet

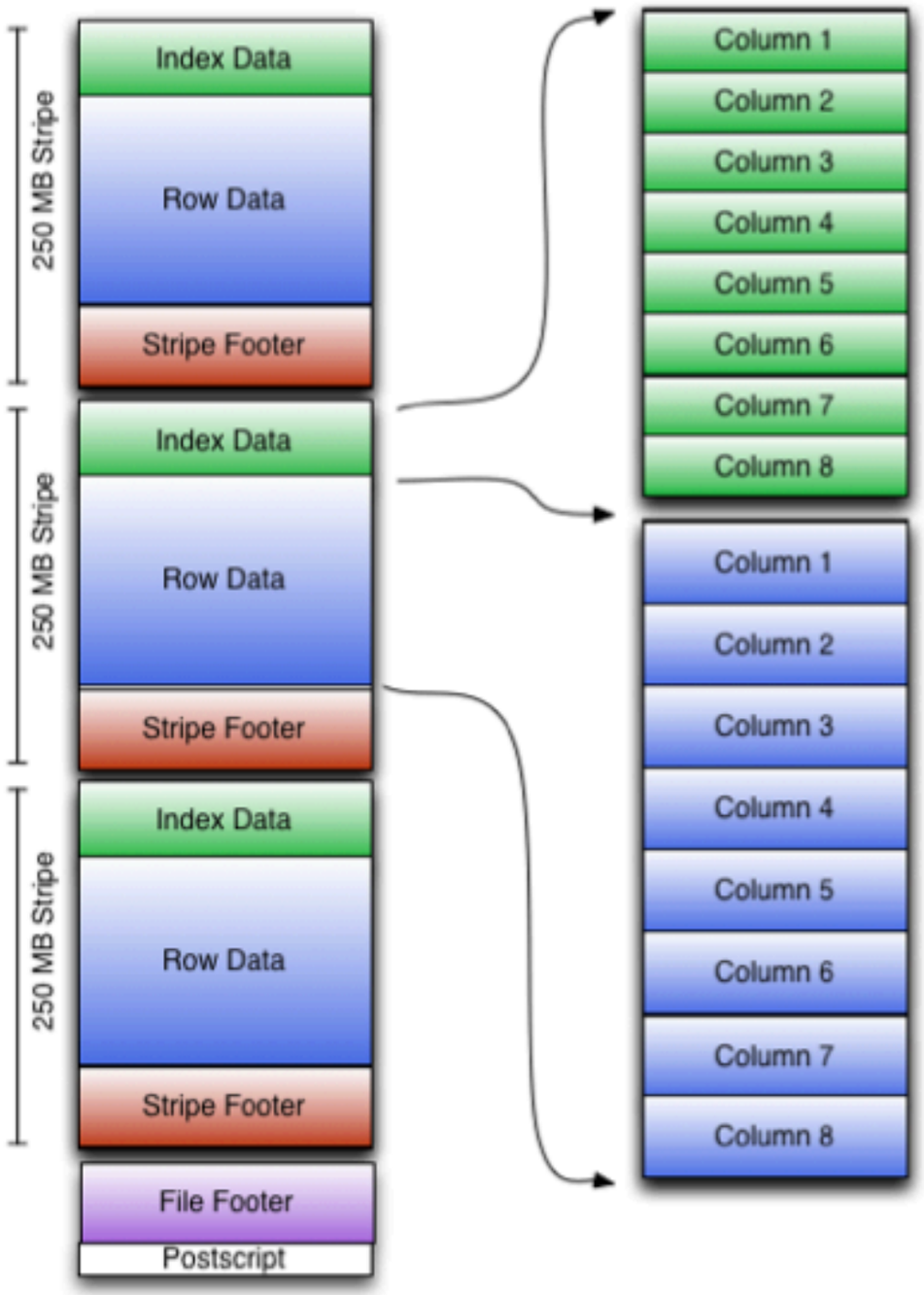
- Is Parquet truly columnar?
- Well, data is not stored in rows …
- In a RowGroup, each ColumnChunk is stored using its own amount of Pages with their own size and encodings
- Row-based lookup over columns is not trivial (Pages are NOT aligned)
- The columnar data is fragmented and not available in a CPU readable format

the fragments are also too big to e.g. do decompression in CPU caches

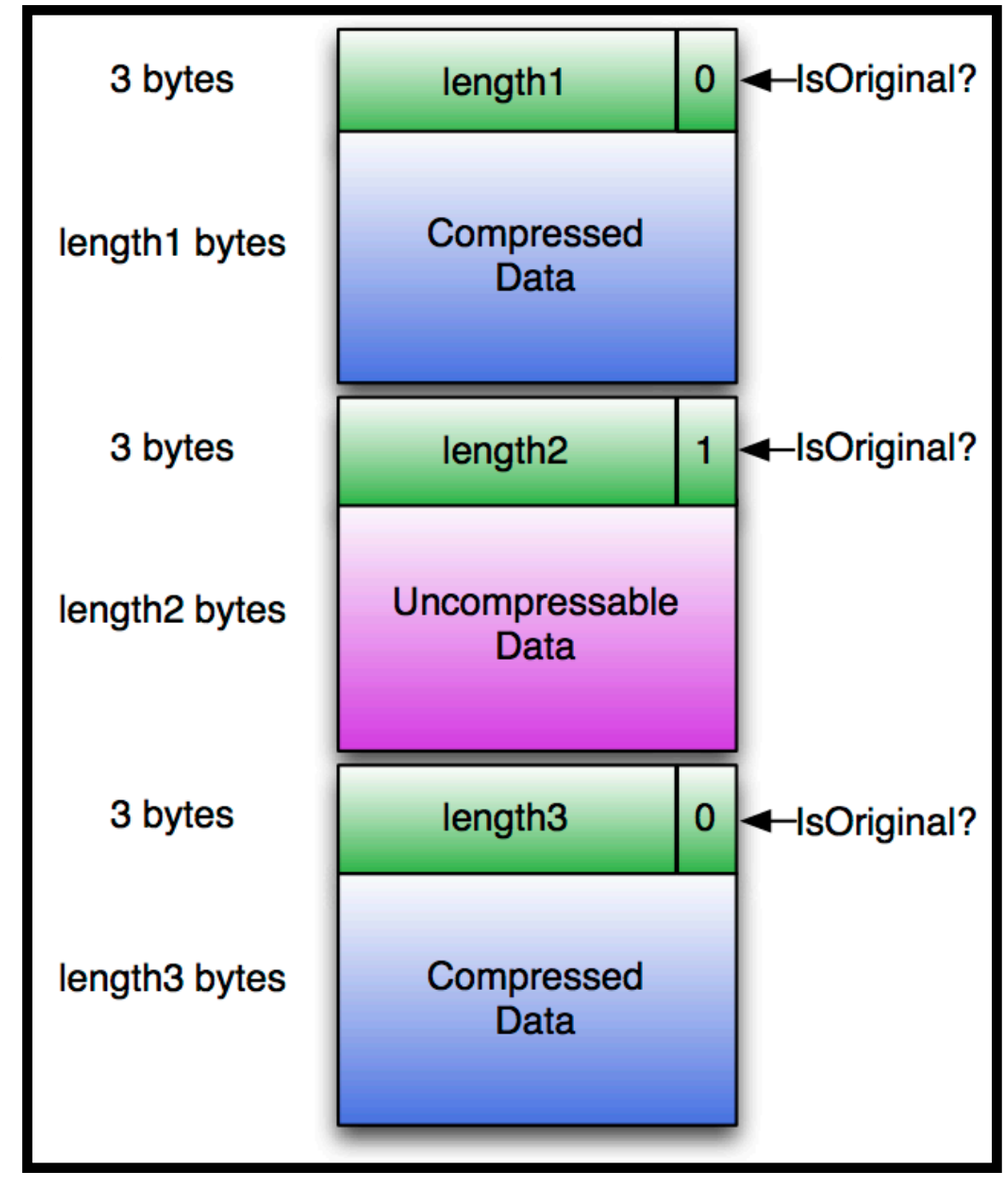
ORC

- Used by Hortonworks Hadoop
- Similar or close to identical to Parquet feature set
- Like Parquet, metadata is found at the back of the file
- Uses Stripe instead of RowGroup
- Uses Stream instead of ColumnChunk
- Uses protobuf instead of Thrift
- Schema nesting is simpler (Parquet uses pseudo intermediate columns)
- Supports encryption and redaction (data for column without enc. key)

ORC structure



like Parquet, a column is written in chunks



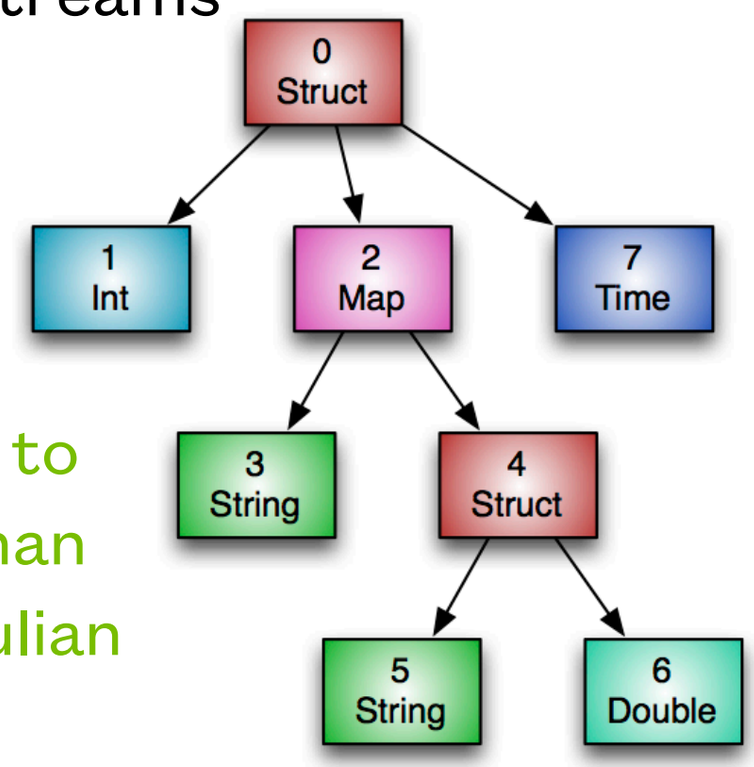
unlike Parquet, the chunks default to 256K and are limited to ~8MB in size

Reading ORC metadata

- Using protobuf, similar to ThriftCompactProtocol
- At the back of the file
- Data types, but also compound types have data streams
- Timestamp type is seconds since 2015-01-01
- Writer encodes its own timezone
- Need to read entire Stripe for Footer with pointers to streams
- Statistics per 10K rows

so a list, is just an array of lengths, simple!

often easier to work with than Parquet's Julian date



ORC streams

- Each column has 1 or more streams
- Each stream can be read individually
- Example, timestamp column:
  - PRESENT, booleans (NULLs)
  - DATA, signed integers (seconds since 2015-01-01)
  - SECONDARY, unsigned integers (nanosecond values)
- Encodings are used like run-length, special tricks to encode trailing 0s, and more

slightly more CPU friendly, at least simpler to implement

unfixed order of streams hurts “streaming” though

Avro

- Non-columnar format
- Stream-based, instead of blocks like Parquet/ORC
- JSON schema as part of Avro binary encoded metadata
- Binary encoding of rows in DataBlocks
- Complex schema definition including “evolution” support
- Object Container files with Binary Encoding

- No metadata, such as statistics over the data

Avro file structure

- Object Container is
  - file metadata, as key-values (Avro map encoding)
    - codec (compression algorithm)
    - schema in Avro JSON schema form
  - data blocks
    - length prefix (to allow skipping)
    - record count
    - trailer 16-byte sync marker (to recover/resume)

Avro schema

yay, finally something we humans can read!

- JSON syntax

inline defined types can be referenced later

```
{
  "type": "record",
  "name": "test",
  "fields" : [
    {"name": "a", "type": "long"},
    {"name": "b", "aliases": [“B”, “bee”], "type": “string”},
    {"type": "enum", "name": "Foo", "symbols": ["A", "B", "C", "D"] },
    {"name": "myfoo", "type": “Foo”},
    {"name": "magic", "type": {"type": "fixed", "name": "Magic", "size": 4}},
    {"name": "serverHash",
     "type": ["null", {"type": "fixed", "name": "MD5", "size": 16}]},
    {"name": “a-decimal”,
     "type": {
              "type": "bytes",
              "logicalType": "decimal",
              "precision": 4,
              "scale": 2
             }
    }
  ]
}
```

lots of options and types make value interpretation complex

Avro data block

cool, but when would you want that? no metadata

- Can be skipped, length prefix
- Can be compressed
- Binary encoding of a set of rows, record count known
- Each row is encoded as value after value
  - values follow the schema
  - nesting, lists, unions, enums, etc. are written directly

reader needs to fully understand the schema (data types and their storage) in order to extract the data

BigData format recap

Parquet and ORC	Avro
binary format, using RowGroups/Stripes of row values	binary format, using DataBlocks of row values
column oriented storage of values allows skipping reading of unnecessary columns	row oriented storage of values requires read of all columns to finish a row
reading data requires to read the **end** of the file first	reading data requires to read the **start** of the file first
value statistics per data vector to allow skipping RowGroups, columns can be skipped	all data must be read
column values encoded with simple to extremely complicated schemes	column values use relatively simple encodings (varint)
scattered reads back and forth	stream oriented (forwards only)

My observations

- Parquet is very complicated
  - and hard to get right
  - documentation is incomplete if it exists at all
  - it appears to be more complicated than it has to be
- ORC is simpler, providing the same functionality
  - better documented
  - tricky at some points thanks to micro optimisations
  - looks as some “lessons learned” from Parquet

My observations (2)

perhaps biased based on intended data usage

- Both Parquet and ORC require a lot of CPU to unpack data values
  - mostly very CPU unfriendly (lots of branches per value)
  - copies of the data to turn them into real values
- I/O patterns are suboptimal for HTTP access
  - first read the end, in order to read any data from the file (at the start)
  - metadata read at end for Parquet is optimistic (size unknown)
  - ORC metadata has notes about max and expected sizes for this (16K)

My observations (3)

- Parquet and ORC are both write-optimised formats (!)
  - metadata at end is to avoid writer to have to keep data in memory
    - HDFS allows write once (stream) only
  - BigData (analytics) expectation is to read more than write
    - IMHO poor design to favour the writer
  - writers determine memory requirements via sizes of RowGroups, Stripes, ColumnChunks, Streams, Pages and blocks.
    - readers processing files in parallel may prefer different requirements
    - ORC seems to keep this in mind, Parquet certainly not

My observations (4)

- Avro is stream and row-based, but uses block units of rows
- Blocks tend to be (much) smaller in size than RowGroups and Stripes
- No statistics or anything, so block skipping capability is rather pointless
- Gentle/efficient with HTTP fetch
- Memory requirements thanks to smaller blocks and row-orientation better manageable by reader

If I could choose

- A format that
  - stores metadata at the front of the file, including statistics or indices
  - stores columnar, values back to back (arrays), no bit-level savings (e.g. PLAIN encoding)
  - in relatively manageable sized blocks based on value size, not rows
    - e.g. rather MBs than GBs
  - uses optional compression on top of the blocks (because they tend to do a pretty good job)

Thank you

Questions?